

Responsiveness with Edge Delivery + Universal Editor

With **Edge Delivery Services (EDS)** and the **Universal Editor (UE)**, *responsiveness is not configured in the editor itself*. It comes from:

1. **Your front-end code (CSS + JS) in GitHub**
2. **The structure of blocks/sections in the HTML** that EDS generates from AEM content
3. **Viewport emulation in UE** – purely for preview, not for layout rules

Example:

Presentation side (EDS + GitHub)

Each block has:

block.js – behavior & decoration

block.css – layout & responsive rules

You define breakpoints and responsive behavior entirely in CSS, e.g.:

```
/* Desktop */
```

```
.cards {
```

```
display: grid;
```

```
grid-template-columns: repeat(3, 1fr);
```

```
gap: 1.5rem;
```

```
}
```

```
/* Tablet */
```

```
@media (max-width: 1199px) and (min-width: 768px) {
```

```
.cards {
```

```
grid-template-columns: repeat(2, 1fr);
```

```
}
```

```
}
```

```
/* Mobile */
```

```
@media (max-width: 767px) {
```

```
.cards {
```

```
grid-template-columns: 1fr;
```

```
}
```

```
}
```

- If your CSS is built with **fluid layouts and media queries**, UE can preview as many “devices” as you care about, even beyond whatever presets Adobe ships.
- To support an additional device class (e.g. very large desktop, small phone):
 - Add the **media query** in your CSS.
 - (Optionally) define a matching preset size in UE configuration so authors have a one-click preview.

There’s no separate “layout mode” like classic AEM responsive grid; authors don’t drag handles per breakpoint – they rely on globally-defined, predictable CSS behavior.

Media Query -Responsive behavior in CSS

In your cards.css:

```
.cards {
```

```
display: grid;
```

```
gap: 1.5rem;
```

```
}
```

```
/* Large Desktop */
```

```
@media (min-width: 1440px) {
```

```
.cards {
```

```
    grid-template-columns: repeat(4, minmax(0, 1fr));  
  }  
}
```

```
/* Desktop */
```

```
@media (min-width: 1200px) and (max-width: 1439px) {  
  .cards {  
    grid-template-columns: repeat(3, minmax(0, 1fr));  
  }  
}
```

```
/* Tablet */
```

```
@media (min-width: 768px) and (max-width: 1199px) {  
  .cards {  
    grid-template-columns: repeat(2, minmax(0, 1fr));  
  }  
}
```

```
/* Mobile */
```

```
@media (max-width: 767px) {  
  .cards {  
    grid-template-columns: 1fr;  
  }  
}
```

Custom section block

You can absolutely create a **custom “section-style” block** for an Edge Delivery page used with **Universal Editor**.

But there are two important constraints:

1. **Every page still needs a top-level default section wrapper** (the basic EDS page structure).
2. Your “custom section” is then implemented as a **block inside that section** (or inside multiple sections), not as a replacement for the core section concept.

So: you don’t replace the default section; you build your own **block that behaves like a section** from a design point of view.

Why you can’t get rid of the “default section”

For Edge Delivery, Adobe standardizes page structure as:

- Page = **one or more sections**
- Each section = optional default content + 0..n blocks

This structure is used consistently for:

- Document-based authoring (Word / Google Docs)
- AEM+Universal Editor authoring

The Edge Delivery runtime, block libraries, and tooling assume that structure; if you tried to “skip” the base section layer, you’d be fighting the platform.

What a “custom section” actually looks like in practice

The typical pattern is:

```
<section class="section [variant classes]">
```

```
<!-- your custom layout block -->
```

```
<div class="my-section-block">
```

```
... content ...
```

```
</div>
```

```
</section>
```

In current EDS guidance:

- **Sections are structural**; they’re the outer container.
- **Blocks are the functional/layout units**.

You generally **do not** create a new HTML `<section>` nested inside another `<section>` just to get a “custom section” – that tends to complicate semantics and styling unnecessarily.

Instead:

- Keep a **single outer** `<section class="section">` adopted from the standard page structure.
- Implement your visual variation as:
 - `class="section section--dark"` on the section itself **and/or**
 - A block like `.my-section-block` inside that section which controls the inner layout.

In UE this often translates to:

- A **Section config** (background color, theme) – sometimes driven by metadata or a small “section decoration” block.
- A **primary layout block** as the “custom section”.

Summary

Can we create a customised section block for our edge delivered page in universal editor?

Yes. You implement it as a **custom block** that lives inside a section and controls layout and styling so that it looks/behaves like a “section”.

Will that custom section have to be included under the default section?

Yes. Every page remains structured as one or more **sections**, and your custom “section block” is **placed inside** those sections. You don’t replace or remove the default section wrapper.